MODULE 1 · FOUNDATIONS

Inspect Your Cluster

A read-only tour of the machinery behind every earlier slide

WHY IT MATTERS

See the architecture, don't just imagine it

Earlier slides described the control plane, the container runtime, the CNI, the CSI, and CoreDNS as diagrams. This lab is the reality check: the same components, found with `kubectl`, on a live cluster.

Every command here is a **get** or a **describe** — nothing is created, changed, or deleted (except one throwaway debug Pod at the end).

BY THE END YOU CAN

- Find control-plane Pods in `kube-system`
- Read a node's runtime & OS from its status
- Spot the CNI and CSI driver Pods
- Trace a DNS lookup through CoreDNS

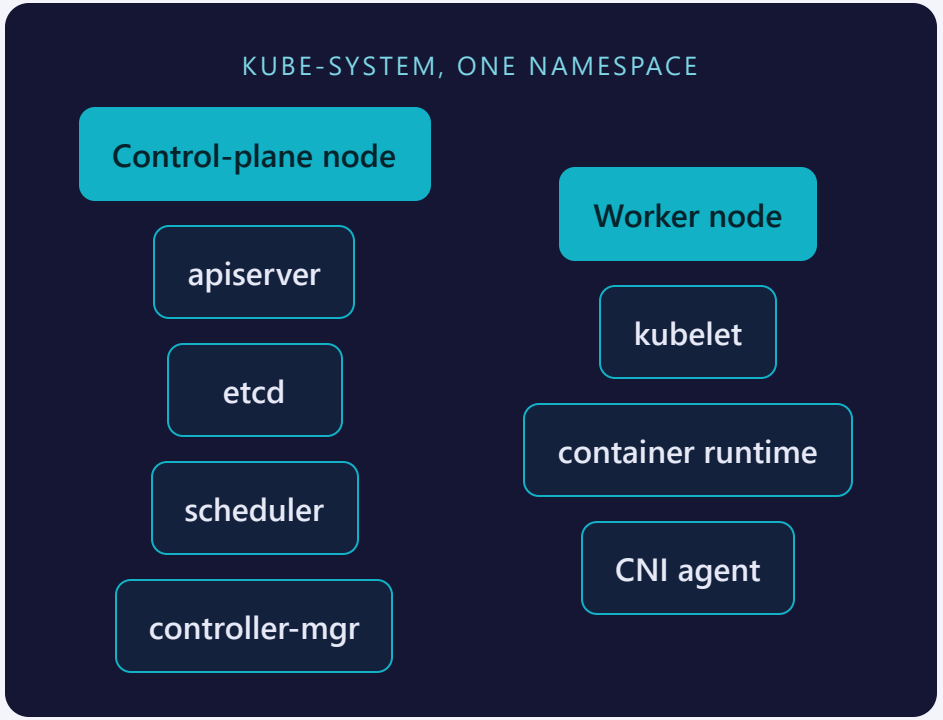
CONCEPT

Two kinds of node, one namespace to check

On a vanilla cluster the control-plane components aren't magic — they're ordinary Pods that the kubelet starts from static manifest files on disk.

Worker nodes carry their own always-on agents: the kubelet itself, the container runtime, and the CNI's per-node agent. Almost all of it — control plane, agents, CNI, CSI, CoreDNS — lives in one namespace: `kube-system`.

Some clusters hide this layer from users entirely. This lab works because ours doesn't.



CONTROL PLANE & NODE AGENTS

Where each piece lives

COMPONENT	RUNS AS	FIND IT WITH
apiserver · etcd · scheduler · controller-manager	static Pod, `tier=control-plane`	`kubectl get pods -n kube-system -l tier=control-plane`
kubelet	host process, not a Pod	`kubectl get node NAME -o jsonpath='{.status.nodeInfo.kubeletVersion}'`
container runtime	host process (e.g. containerd)	`CONTAINER-RUNTIME` column of `kubectl get nodes -o wide`
kube-proxy (or its eBPF replacement)	DaemonSet Pod, `k8s-app=kube-proxy`	`kubectl get pods -n kube-system -l k8s-app=kube-proxy`

WATCH FOR

No kube-proxy Pods is not broken — some CNIs replace it entirely with an eBPF dataplane.

THE ADD-ONS THAT MAKE IT USABLE

Pod IPs and volumes don't appear on their own

LAYER	WHAT IT DOES	FIND IT WITH
CNI plugin	Assigns every Pod its IP from the Pod CIDR	`kubectl get pods -n kube-system -o wide` (grep the CNI's name)
CSI driver	Registered driver object the provisioner uses	`kubectl get csidrivers`
StorageClass	Names the provisioner new PVCs will use	`kubectl get storageclass`

TWO OBJECTS, ONE RELATIONSHIP

A StorageClass's `provisioner` field names a CSI driver — but some default classes use a simpler, non-CSI provisioner. Check both objects, not just one.

SEE IT

A Service name, resolved

WHERE A LOOKUP ACTUALLY GOES


```
$ kubectl exec dnsutil -n training -- cat /etc/resolv.conf   # search domains, ndots
$ kubectl exec dnsutil -n training -- nslookup kubernetes.default.svc.cluster.local
```

Every Pod's `resolv.conf` points at the `kube-dns` Service's ClusterIP; the CoreDNS Pods behind it answer using rules from the `coredns` ConfigMap's Corefile.

GOTCHAS — THE ONES THAT BITE

Where people trip

- **Empty `get csidrivers`.** A default StorageClass can point at a simple, non-CSI provisioner — absence of a driver object doesn't mean no storage.
- **No kube-proxy Pods.** Not a fault — some CNI eBPF dataplanes replace kube-proxy outright.
- **Reading only `get nodes`.** The default columns hide the runtime — always add `-o wide`.
- **Assuming DNS is slow/broken.** `ndots:5` makes short names try every search suffix first — normal, not a bug.

YOUR INSPECTION TOOLKIT

Commands to keep at hand

```
$ kubectl get pods -n kube-system -l tier=control-plane   # apiserver, etcd, scheduler…
$ kubectl get nodes -o wide                               # runtime, OS, kubelet version
$ kubectl get pods -n kube-system -o wide | grep -i cni   # CNI agent Pods, per node
$ kubectl get csidrivers                                  # registered CSI drivers
$ kubectl get storageclass                                # provisioner behind each class
$ kubectl get pods,svc -n kube-system -l k8s-app=kube-dns # CoreDNS Pods + ClusterIP
$ kubectl exec POD -- nslookup NAME.default.svc.cluster.local
```

Tip: `describe` node for the full picture — labels, capacity, conditions, and every Pod scheduled there.

RECAP

Your cluster in one breath

- `kube-system` holds the control plane, node agents, CNI, CSI, and CoreDNS — nothing is hidden here
- `-o wide` reveals the container runtime and node IP; `describe` reveals the rest
- CNI assigns Pod IPs; CSI drivers back StorageClasses; CoreDNS answers Service names
- A Pod's `resolv.conf` + the `kube-dns` Service is the whole DNS story

HANDS-ON

Now run `Lab 1.2`

`labs/1.2/`

Next: **2.1 — Pods**